```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

3. Tutorial - Constructors & Methods

Kuzey Aydınhan | 21. November 2025

Contents

1. Overview

2. Common Expressions

3. Scanner

4. Methods

5. JavaDoc

Information

Important

- 2nd Homework is out (in ILIAS)
- Sign up for Übungsschein in Campus System until 10.12. 12:00
- There is no tutorial on 19.12. (1 week before xmas)
- There will be a Saalübung on 02.02.

Last Week

Data Types

Common data type operators.

- String Operators
- Artihmetic Operators
- Logarithmic Operators
- Difference between a++ vs ++a
- Difference between && vs &

Last Week

Control Structures

Control structures used in Java.

- `If, Switch` Expressions
- `For, While, do-While` structures
- `Continue, Break`

Last Week

If Structure

Correct:

```java
if (string.equals("this")) {

} else if (string.equals("that")) {

} else if (string.equals("both")) {

} else {

}

```

False:

```java
if (string.equals("this")) {

} else {
    if (string.equals("that")) {

    } else {
        if (string.equals("both")) {

        } else {

        }
    }
}

```

Last Week

If Structure

Correct:

```java
if (string.equals("this")) {

} else {
    if (integer > 0) {

    } else if (integer < 0) {

    } else {

    }
}

```

False:

```java
if (string.equals("this")) {

} else if (integer > 0) {

} else if (integer < 0) {

} else {

}

```

Motivation

- Common expressions: `instanceOf & (type)`
- `Scanner` usage
- Methods and signatures
- Constructor methods

Inline If

- Already seen last week
- `__ ? __ : __`

```java
boolean isTrue = true;
int number = isTrue ? 5 : 10;  // = 5

```

instanceof

```java
Integer number = 5;
boolean isInteger = number instanceof Integer;  // = true

int primitiveNumber = 5;
boolean invalidExample = primitiveNumber instanceof Integer;  // Exception thrown

Integer nullable = null;
boolean isObject = nullable instanceof Integer;  // = false

```

- Type check
- Returns `boolean`
- Only works with non-primitives
- Always false if left-hand side is `null`

Careful

Never use `instanceof` outside `equals` methods.
Guideline: https://sdq.kastel.kit.edu/programmieren

Type Casting

Idea:

- Force a type
- Convert from one type to another
  - Point number → Integer
  - Integer → Point number
  - **Invalid:** Integer → Boolean

```java
double price = 15.95;
int roundedPrice = (int) price;  // = 15

char asciiCharacter = 'A';
int asciiNumber = (int) asciiCharacter;  // = 65

```

Type Casting

Risk

- When going from large → small
- Bits/information lost
- Overflow risk

```java
long bigValue = 192749295273494L;
int smallValue = (int) bigValue;  // Unknown due to information loss

```

Scanner

Problem

How to get inputs into program?

- Command-line arguments: `void main(String[] args)`
- `Scanner` inputs

- Command-line arguments: altering program execution/passing config file path etc.
- Scanner: direct input while program running
- Common usage: program commands (`"quit", "move" etc.`)

```java
import java.util.Scanner;

static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("Enter something:");
    String input = scanner.nextLine();

    scanner.close();
}

```

Scanner

- `String` input: `Scanner::nextLine`
- Can also use numbers:
  - `Scanner::nextInt`
  - `Scanner::nextDouble`

```java
import java.util.Scanner;

static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("Enter your age:");
    int input = scanner.nextInt();

    scanner.close();
}

```

Scanner

Important

- Must have only one scanner instance!
- Can't have multiple
- Always close the scanner

```java
import java.util.Scanner;

static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Never do this!
    Scanner secondScanner = new Scanner(System.in);

    System.out.println("Enter your age:");
    int input = scanner.nextInt();

    // Always close
    scanner.close();
}

```

Scanner

Usage

```java
static void main(String[] args) {
    Scanner scanner = new Scanner();

    while (scanner.hasNextLine()) {
        String input = scanner.nextLine();
        // ...
    }

    scanner.close();
}

```

- Waits for input: `Scanner::hasNextLine`
- Used with `while` loops
- Don't forget to break the loop!
- Can create command structure using with `enums`

Exercise 1

```java
enum Command {
    QUIT, SAY_HI
}

class Main {
    public static void main(String[] args) {
        // TODO: Implement me
    }

    static void correctAnswer() {
        System.out.println("Correct answer");
    }
}

```

- **Take** single command-line argument at the start
- Keep asking the user for input
- If the given input matches a command execute it
  - `QUIT`: Stops the loop and exits
  - `SAY_HI`: Prints out "Hello World"
- If the given input matches the initially passed command-line argument, execute `correctAnswer`

Useful

`Enum::name` returns the `enum` as `String`

Methods

- Classes: Same elements
- Attributes: Properties
- Methods: Actions

```java
class Car {
    // Attributes
    Color color;
    String name;
    String model;

    // Methods
    void turnOn () {
        // ...
    }
}

```

Methods

Return Values

```java
int calculate(int a, int b) {
    // ...

    return result;

    // Unreachable code
}

void move(int x, int y) {
    // ...

    if (expression) {
        return;  // Can be used for early exits
    }

    // ...
}

```

- Basically can return any type (primitive/non-primitive)
- Can also return nothing (`void` keyword)
- Must use `return` keyword, except in **`void`**

Methods

Parameters

```
int calculate(int a, int b) { ... }

void move(int x, int y, Car car) {
    // ...

    x = 5;  // Never do this

    // ...
    car.x = x;
}

```

- Can pass in values to methods
- No limitation
- Parameters will get copied!
  - Cannot & must not overwrite parameters
  - Only the references of classes will get passed in

Local Variables

- Only exists inside of method
- Can be
  - initialized the default way
  - initialized with `null` (non-primitive)
  - late initialized (using references)

```java
void bar() {
    int default = 15;

    Integer y = null;

    int x;
    x = 20;

    // ...
}

```

Constructor Methods

- Used for creating instances
- Setting attributes
- Doesn't have any return type
- Returns the created instance itself
- Matches the class name
- `this` keyword for accessing the **current** instance

```java
class Car {
    String name;
    Color color;
    String model;

    Car(String name, Color color, String model) {
        this.name = name;
        this.color = color;
        this.model = model;
    }
}

```

Constructor Methods

Instantiaton

- Used with new keyword
- Creates an instance

```java
class Main {
    public static void main(String[] args) {
        Car car = new Car("My Mercedes", Color.BLACK, "c200d");

        // Unnecessary:
        car.name = "My Mercedes";
        // ...
    }
}

```

Default Constructor

```java
class Car {
    String name;
}

class Main {
    public static void main(String[] args) {
        // Default constructor
        Car car = new Car();
        car.name = "My Mercedes";
    }
}

```

- There is a default constructor for each class
- Even if you dont specifically define it
- However it gets overwritten, if you put any other constructors

Enum Constructor

```java
enum Color {
    BLACK(1,1,1),
    BLUE(0,0,1),
    GREEN(0,1,0),
    WHITE(0,0,0);

    final double red;
    final double green;
    final double blue;

    Color(double red, double green, double blue) {
        this.red = red;
        this.green = green;
        this.blue = blue;
    }

    double getRed() {
        return this.red;
    }

    // ...
}

```

Method Overloading

```java
class Car {
    String name;

    double x;
    double y;

    void move(int x, int y) {
        this.x += x;
        this.y += y;
    }

    void move(double x) {
        this.x += x;
    }
}

```

- Can have same method with different parameters
- Name & return type must be same
- Useful when similar functionality
- Can also be used with constructor methods

Don't!

Do not implement functionally different methods using method overload! If using method overload, the methods should do similar -if not same- things

Getters & Setters

- Will repeat when talking about encapsulation
- Good measure to use `getters/setters` for attributes
- **Getter:** return attribute
- **Setter:** set attribute
- Can also add functionality as validation etc.

```java
class Plane {
    String model;
    String name;

    void setName(String name) {
        this.name = name;
    }

    String getName() {
        return this.name;
    }
}

```

Naming Convention

- **Getters:** `getAttribute()`
- **Setters:** `setAttribute(Type attribute)`

Static Methods

```java
class Student {
    String name;
    static int studentCount = 0;

    Student(String name) {
        this.name = name;
        studentCount++;
    }

    static int getStudentCount() {
        return studentCount;
    }

    static void foo() {
        System.out.println("Fooooo");
        this.name;  // Can’t access -> exception
        this.boo();  // Also no access
    }
}


```

- Can be accessed without any instances of class
- Can’t access class **attributes or methods**
- → `this` keyword not allowed
- Only access to other `static` attributes/methods
- Useful for helper methods (next week)
- `main` always static

```java
public static void main(String[] args) {
    int studentCount = Student.getStudentCount();  // = 0
    Student student = new Student("Maria");

    Student.getStudentCount();  // = 1
}

```

Exercise 2

Description

There is a swimming pool system:

- Pool will hold information on each swimming session
- Each session has a capacity of 25 students
- Each session will cost 2€
- Each student has an amount of cash

Implement:

- Three constructor methods for sessions:
  - Takes `sessionId` and `capacity`
  - Takes `sessionId`, `capacity` and `cost`
  - Default constructor
- `getSession`: Returns the session that matches the passed `sessionId`, returns `null` if no such session available
- `getAllStudents`: Returns all of the `studentIds` in a session
- `addStudent`: Adds a student to session if session is not full, otherwise returns `false`

Exercise 2

Classes

```java
class Session {
    String sessionId;
    int capacity = 25;
    int currentAmount = 0;
    double cost = 2;
    Student[] students = new Student[capacity];

    // TODO: Implement me
}

```

```java
class Pool {
    Session[] sessions;

    // TODO: Implement me
}

class Student {
    String studentId;
    String name;
    double cashAmount;
}

```

JavaDoc

Documentation

- Documenting methods/classes
- Explain -in short- what it does
- Dont need to document private methods (more next week)

- `@author`: Author of method/class (your u-kürzel)
- `@param`: What that method parameter is for
- `@return`: What the method returns
- `@throws`: If any exception can be thrown and why
- `@version`: Version of file (not used much)

Programming Wiki

You can find more information regarding topic under: https://sdq.kastel.kit.edu/programmieren/JavaDoc

JavaDoc

Example

```
\**
 * Represents a car vehicle.
 *
 * @author uxxxx
 * @version 1.0.0.
 **\
class Car {
    String name;

    \**
     * Simulates car driving forward.
     *
     * @param person The person driving this car
     * @return true if car does not hit a wall
     * @throws IllegalArgumentException If the passed in person is null
     * @author uxxxx
     **\
    public boolean driveCar(Person person) {
        // ...
    }
}

```

Conclusion

- `instanceof` returns boolean & type check
- We can convert between types by `(type) cast`
- Scanner usage for inputs
- Don't forget to close scanner!
- Method signatures
- Constructor methods
- Non-Trivial enums (with constructors)
- Method overloading
- Static methods
- JavaDocs

Remember

Important

Homework 2 already out in ILIAS

Solution

Exercise 1

```java
public static void main(String[] args) {
    if (args.length != 1) {
        return;
    }
    String initial = args[0];

    Scanner scanner = new Scanner(System.in);
    while (scanner.hasNextLine()) {
        System.out.println("Enter something:");
        String line = scanner.nextLine();

        if (line.equals(Command.QUIT.name())) {
            break;
        } else if (line.equals(Command.SAY_HI.name())) {
            System.out.println("Hello World");
        } else if (line.equals(initial)) {
            correctAnswer();
        }
    }

    scanner.close();
}

```

Solution

Exercise 2

```java
class Pool {
    Session[] sessions;

    Session getSession(String sessionId) {
        if (sessionId == null) {
            return null;
        }

        for (Session session : sessions) {
            if (sessionId.equals(session.sessionId)) {
                return session;
            }
        }
        return null;
    }
}

```

Solution

Exercise 2

```java
class Session {
    String sessionId;
    int capacity = 25;
    int currentAmount = 0;
    double cost = 2;
    Student[] students = new Student[capacity];

    Session(String sessionId, int capacity, double cost) {
        this.sessionId = sessionId;
        this.capacity = capacity;
        this.students = new Student[capacity];
        this.cost = cost;
    }

    Session(String sessionId, int capacity) {
        this.sessionId = sessionId;
        this.capacity = capacity;
        this.students = new Student[capacity];
    }


```

```java
    Session() {
        // Default constructor
    }

    String[] getAllStudents() {
        String[] studentIds = new String[students.length];
        for (int i = 0; i < students.length; i++) {
            studentIds[i] = students[i].studentId;
        }
        return studentIds;
    }

    boolean addStudent(Student student) {
        if (currentAmount < capacity) {
            students[currentAmount] = student;
            currentAmount++;
            return true;
        }
        return false;
    }
}

```